

Day 4 - DSA Array Prefix Sum, Difference array

ITSRUNTYM

1. What is the Prefix Sum Pattern?

The **Prefix Sum** of an array is a technique where you create a new array (or use a data structure) to store cumulative sums of elements from the start up to each index. Formally, for an array `arr[]` of size `n`, its prefix sum array `prefix[]` is defined as:

$$\text{prefix}[i] = \text{arr}[0] + \text{arr}[1] + \dots + \text{arr}[i]$$

for $0 \leq i < n$

2. Why use Prefix Sum?

Prefix sums help quickly compute the sum of elements in any contiguous subarray in **$O(1)$** time after an **$O(n)$** preprocessing step.

- **Without prefix sums:** Sum of elements from index `i` to `j` is computed by looping from `i` to `j`, which is **$O(j - i + 1)$** .
- **With prefix sums:** Sum of elements from index `i` to `j` is:

$$\text{sum}(i, j) = \text{prefix}[j] - \text{prefix}[i - 1]. \quad (i > 0)$$

or simply

$$\text{prefix}[j] \quad (i = 0)$$

which is **$O(1)$** .

3. How to Compute Prefix Sum?

Given an array `arr` of size `n`:

```
int[] prefix = new int[n];
prefix[0] = arr[0];
for (int i = 1; i < n; i++) {
    prefix[i] = prefix[i - 1] + arr[i];
}
```

4. Where can Prefix Sum be applied?

Prefix sums are useful in many types of problems such as:

- **Range sum queries:** Quickly compute sum of elements in subarrays multiple times.
- **Number of elements in a range** satisfying a condition.
- **Finding subarrays with a certain sum.**
- **2D prefix sums** for matrix sub-rectangle sums.
- **Difference arrays** and range update queries.
- Problems involving **cumulative frequencies**, histograms, or quick summation checks.

5. How to identify problems where Prefix Sum applies?

Look for questions that:

- Ask for **sum of elements in a range/subarray** multiple times.
- Need **fast repeated sum queries** after an initial array is given.
- Need to **find number of subarrays** satisfying certain sum-related properties.
- Involve **checking sums quickly** without recalculating sums for overlapping parts.
- Deal with **prefix-based conditions**, such as count of elements or cumulative constraints.

6. Benefits of Prefix Sum Pattern

- Reduces repeated work in sum calculations.
- Transforms $O(n^2)$ range sum queries into $O(n)$ preprocessing + $O(1)$ query.
- Simplifies problem logic by leveraging precomputed cumulative data.
- Helps in solving problems related to subarray sums, histogram calculations, and range queries efficiently.

2. Difference Array Pattern

Definition

The **Difference Array** allows you to perform **range update operations in constant time ($O(1)$)**.

Instead of updating all elements in a range one by one, we update only **two positions** in a difference array and apply a prefix sum at the end.

Core Idea

We construct a difference array `diff[]` where:

```
diff[i] = arr[i] - arr[i-1] (for i > 0)
diff[0] = arr[0]
```

To increment all values in a range l to r by x:

```
diff[l] += x
diff[r+1] -= x (if r+1 is within bounds)
```

After all updates, we take prefix sum of diff[] to get final arr[].

Example

Problem:

You have an array of size n initialized with 0. Perform q operations: each adds x to all elements from index l to r.

```
int[] arr = new int[6]; // n = 6
int[] diff = new int[6];

// Operation 1: Add 10 to range [1, 3]
diff[1] += 10;
diff[4] -= 10;

// Operation 2: Add 5 to range [2, 5]
diff[2] += 5;
diff[6] -= 5; // out of bounds, skip

// Final array using prefix sum
arr[0] = diff[0];

for (int i = 1; i < 6; i++) {
    arr[i] = arr[i-1] + diff[i];
}
```

Output:

```
arr = [0, 10, 15, 15, 5, 5]
```

LeetCode 370. Range Addition

Problem Statement:

You are given an integer length and a list of update operations. Initially, an array of size length is filled with all 0s.

Each update operation is represented as a triplet: [startIndex, endIndex, increment].

You have to apply all the operations and return the final modified array.

Example:

Input: length = 5, updates = [[1,3,2],[2,4,3],[0,2,-2]]

Output: [-2,0,3,5,3]

Explanation:

Initial array: [0, 0, 0, 0, 0]

After [1,3,2] => [0, 2, 2, 2, 0]

After [2,4,3] => [0, 2, 5, 5, 3]

After [0,2,-2] => [-2, 0, 3, 5, 3]

Optimized Approach: Difference Array

Intuition:

Instead of applying the update for all elements in the range [startIndex, endIndex], we just update:

```
arr[startIndex] += val;  
arr[endIndex+1] -= val; // if within bounds
```

Then, take prefix sum over arr[] to get final result.

Java Code (Optimized)

```
public int[] getModifiedArray(int length, int[][] updates) {  
    int[] res = new int[length];  
  
    // Step 1: Apply difference array update  
    for (int[] update : updates) {  
        int start = update[0], end = update[1], val = update[2];  
        res[start] += val;  
        if (end + 1 < length) {  
            res[end + 1] -= val;  
        }  
    }  
}
```

```

    }

// Step 2: Take prefix sum to get final result
for (int i = 1; i < length; i++) {
    res[i] += res[i - 1];
}
return res;
}

```

Time Complexity:

- Updates: $O(q)$
- Final prefix sum: $O(n)$
- Total: **$O(n + q)$** — very efficient

When to Use Difference Array

- You have **multiple range update queries**
- Each update affects a **subarray** or **range of elements**
- You want to optimize brute-force $O(n \cdot q)$ into $O(q + n)$

Common Questions Using Difference Array

1. [Medium] Range update in array
2. [Hard] Apply multiple range increment queries
3. [Medium] Number of overlapping intervals
4. [Hard] Maximum value after range updates (HackerRank)

Comparison: Prefix Sum vs Difference Array

Feature	Prefix Sum	Difference Array
Purpose	Fast range sum queries	Fast range update queries
Time for query/update	$O(1)$ query after $O(n)$ preprocess	$O(1)$ update after $O(1)$ diff update
Use case	Subarray sums	Range modifications
Preprocessing	Prefix sum array	Difference array + prefix after all ops

How to Decide Which to Use

Situation	Use
You have to answer sum of elements in a range frequently	Prefix Sum

You have to update ranges frequently , then output final array	Difference Array
Problem involves subarray sums or counting prefix matches	Prefix Sum
Problem says: " Add x to all elements from i to j "	Difference Array

Summary

Pattern	Purpose	Best For
Prefix Sum	Fast subarray sum queries	Range sum problems
Difference	Fast range updates	Range addition / modification problems

#	Problem Name	LeetCode Link
1	Range Sum Query - Immutable	https://leetcode.com/problems/range-sum-query-immutable/
2	Subarray Sum Equals K	https://leetcode.com/problems/subarray-sum-equals-k/
3	Maximum Size Subarray Sum Equals k	https://leetcode.com/problems/maximum-size-subarray-sum-equals-k/
4	Find Pivot Index	https://leetcode.com/problems/find-pivot-index/
5	Count Number of Nice Subarrays	https://leetcode.com/problems/count-number-of-nice-subarrays/
6	Binary Subarrays With Sum	https://leetcode.com/problems/binary-subarrays-with-sum/
7	Minimum Size Subarray Sum	https://leetcode.com/problems/minimum-size-subarray-sum/

8	Maximum Average Subarray I	https://leetcode.com/problems/maximum-average-subarray-i/
9	Number of Subarrays with Bounded Maximum	https://leetcode.com/problems/number-of-subarrays-with-bounded-maximum/
10	Prefix and Suffix Search	https://leetcode.com/problems/prefix-and-suffix-search/
11	Continuous Subarray Sum	https://leetcode.com/problems/continuous-subarray-sum/
12	Longest Subarray of 1's After Deleting One Element	https://leetcode.com/problems/longest-subarray-of-1s-after-deleting-one-element/
13	Number of Subarrays with Odd Sum	https://leetcode.com/problems/number-of-sub-arrays-with-odd-sum/description/
14	Longest Subarray With Sum at Most K	https://leetcode.com/problems/longest-subarray-with-sum-at-most-k/
15	Find the Longest Balanced Substring of Parentheses	https://leetcode.com/problems/longest-valid-parentheses/